

面向对象第四天：

回顾：

1. 抽象方法：
 - abstract修饰，只有方法的定义，没有具体的实现(连{}都没有)
2. 抽象类：
 - abstract修饰，包含抽象方法的类必须是抽象类，不能被实例化
 - 需要被继承，派生类：重写所有抽象方法
 - 意义：代码复用，可以包含抽象方法，为派生类提供统一的入口(名字一致)，强制必须重写
3. 接口：
 - interface定义，抽象方法，不能被实例化，
 - 需要被实现，实现类：必须重写所有抽象方法
 - 一个类可以实现多个接口，用逗号分隔。若又继承又实现时，应先继承后实现
 - 接口可以继承接口
4. 引用类型数组：
 - 区别1：给引用类型数组的元素赋值时，需要new个对象
 - 区别2：若想访问引用类型数组元素的属性/方法，需要通过元素去打点来调用

精华笔记：

1. 多态：多种形态
 - 向上造型/自动类型转换：
 - 超类型的引用指向派生类的对象
 - 能点出来什么，看引用的类型-----这是规定，记住它
 - 向下转型/强制类型转换，成功的条件只有如下两种：
 - 引用所指向的对象，就是该类型
 - 引用所指向的对象，实现了该接口或继承了该类
 - 强转时若不符合如上条件，则发生ClassCastException类型转换异常

建议：在强转之前先通过instanceof来判断引用的对象是否是该类型

注意：instanceof返回boolean结果，它为true的条件就是强转成功的条件

何时需要强转：若想访问的属性/行为在超类中没有，则需要强制类型转换
2. 成员内部类：应用率低，了解
 - 类中套类，外面的称为外部类，里面的称为内部类
 - 内部类通常只服务于外部类，对外不具备可见性
 - 内部类对象通常在外部类中创建
 - 内部类可以直接访问外部类的成员，在内部类中有个隐式的引用指向创建它的外部类对象

隐式的引用：外部类名.this

- 何时用：若A类(Baby)只让B类(Mama)用，并且A类(Baby)还想访问B类(Mama)的成员时，可以设计成员内部类

3. 匿名内部类：应用率高，掌握

- 何时用：若想创建一个派生类的对象，并且对象只创建一次，可以设计为匿名内部类，可以大大简化代码
- 注意：匿名内部类中不能修改外面局部变量的值
- 小面试题：
 - 问：内部类有独立的.class吗？
 - 答：有

4. package和import：

- package：声明包
 - 作用：避免类的命名冲突
 - 规定：同包中的类不能同名，但不同包中的类可以同名。
 - 类的全称：包名.类名。包名常常有层次结构
 - 建议：包名所有字母都小写
- import：导入类
 - 同包中的类可以直接访问，但不同包中的类不能直接访问，若想访问：
 - 先import导入类，再访问类-----建议
 - 类的全称-----太繁琐、不建议

笔记：

1. 多态：多种形态

- 向上造型/自动类型转换：
 - 超类型的引用指向派生类的对象
 - 能点出来什么，看引用的类型-----这是规定，记住它
- 向下转型/强制类型转换，成功的条件只有如下两种：
 - 引用所指向的对象，就是该类型
 - 引用所指向的对象，实现了该接口或继承了该类
- 强转时若不符合如上条件，则发生ClassCastException类型转换异常

建议：在强转之前先通过instanceof来判断引用的对象是否是该类型

注意：instanceof返回boolean结果，它为true的条件就是强转成功的条件

何时需要强转：若想访问的属性/行为在超类中没有，则需要强制类型转换

```
public abstract class Animal {  
    String name;  
    int age;  
    String color;  
    Animal(){  
    }  
}
```

```

Animal(String name,int age,String color){
    this.name = name;
    this.age = age;
    this.color = color;
}

void drink(){
    System.out.println(color+"色的"+age+"岁的"+name+"正在喝水...");
}
abstract void eat();
}
public interface Swim {
    /** 游泳 */
    void swim();
}
public class Dog extends Animal implements Swim {
    Dog(){
    }
    Dog(String name,int age,String color){
        super(name,age,color);
    }

    void lookHome(){
        System.out.println(color+"色的"+age+"岁的狗狗"+name+"正在看家...");
    }
    void eat(){
        System.out.println(color+"色的"+age+"岁的狗狗"+name+"正在吃肯头...");
    }
    public void swim(){
        System.out.println(color+"色的"+age+"岁的狗狗"+name+"正在游泳...");
    }
}
public class Fish extends Animal implements Swim {
    Fish(){
    }
    Fish(String name,int age,String color){
        super(name,age,color);
    }

    void eat(){
        System.out.println(color+"色的"+age+"岁的小鱼"+name+"正在吃小虾...");
    }
    public void swim(){
        System.out.println(color+"色的"+age+"岁的小鱼"+name+"正在游泳...");
    }
}
public class Chick extends Animal {
    Chick(){
    }
    Chick(String name,int age,String color){
        super(name,age,color);
    }
    void layEggs(){

```

```

        System.out.println(color+"色的"+age+"岁的小鸡"+name+"正在下蛋...");
    }
    void eat(){
        System.out.println(color+"色的"+age+"岁的小鸡"+name+"正在吃小米...");
    }
}

public class Master {
    void feed(Animal animal){ //喂动物，将范围扩大了，所有动物都可以
        animal.eat();
    }
}

package ooday04;
/** 多态的测试类 */
public class PolymorphicDemo {
    public static void main(String[] args) {
        /*
        //演示强转成功的条件
        Animal o = new Dog("小黑",2,"黑"); //向上造型
        Dog g = (Dog)o; //引用o所指向的对象，就是Dog类型
        Swim s = (Swim)o; //引用o所指向的对象，实现了Swim接口
        //Fish f = (Fish)o; //运行时会发生ClassCastException类型转换异常

        System.out.println(o instanceof Dog); //true
        System.out.println(o instanceof Swim); //true
        System.out.println(o instanceof Fish); //false
        if(o instanceof Fish){ //false
            Fish f = (Fish)o;
        }
        */

        //演示向上造型(多态)的第2点应用：
        //Animal o = new Animal(); //编译错误，抽象类不能被实例化
        Animal[] animals = new Animal[5];
        animals[0] = new Dog("小黑",2,"黑"); //向上造型
        animals[1] = new Dog("小白",1,"白");
        animals[2] = new Fish("小金",1,"金");
        animals[3] = new Fish("小花",2,"花");
        animals[4] = new Chick("小灰",3,"灰");

        for(int i=0;i<animals.length;i++){ //遍历所有动物
            System.out.println(animals[i].name); //输出每个动物的名字
            animals[i].drink(); //每个动物喝水
            animals[i].eat(); //每个动物吃饭
            if(animals[i] instanceof Dog){
                Dog dog = (Dog)animals[i];
                dog.lookHome();
            }
            if(animals[i] instanceof Chick){
                Chick chick = (Chick)animals[i];
                chick.layEggs();
            }
            if(animals[i] instanceof Swim){ //适用于所有实现Swim接口的(会游泳的)

```

```

        Swim s = (Swim)animals[i];
        s.swim();
    }
}
/*
//演示向上造型(多态)的第1点应用:
Master master = new Master();
Dog dog = new Dog("小黑",2,"黑");
Chick chick = new Chick("小花",3,"花");
Fish fish = new Fish("小金",1,"金");
master.feed(dog); //在传参的同时, 系统自动做了向上造型
master.feed(chick);
master.feed(fish);
*/

/*
//演示向上造型:
Dog o1 = new Dog("小黑",2,"黑");    //狗是狗
//o1.name/age/color/drink()/eat()/swim()/lookHome();
Animal o2 = new Dog("小黑",2,"黑"); //狗是动物----向上造型
//o2.name/age/color/drink()/eat()
Swim o3 = new Dog("小黑",2,"黑");    //狗会游泳-----向上造型
//o3.swim();
*/
}
}

```

2. 成员内部类：应用率低，了解

- 类中套类，外面的称为外部类，里面的称为内部类
- 内部类通常只服务于外部类，对外不具备可见性
- 内部类对象通常在外部类中创建
- 内部类可以直接访问外部类的成员，在内部类中有个隐式的引用指向创建它的外部类对象

隐式的引用：外部类名.this

- 何时用：若A类(Baby)只让B类(Mama)用，并且A类(Baby)还想访问B类(Mama)的成员时，可以设计成员内部类

```

package ooday04;
//成员内部类
public class InnerClassDemo {
    public static void main(String[] args) {
        Mama m = new Mama();
        //Baby b = new Baby(); //编译错误, 内部类对外不具备可见性
    }
}

class Mama{ //外部类
    String name;
    void create(){
        Baby b = new Baby(); //正确, 内部类对象通常在外部类中创建
    }
}

```

```

    }
    class Baby{ //成员内部类
        void show(){
            System.out.println(name); //简写
            System.out.println(Mama.this.name); //完整写法, Mama.this指代外部类对象
            //System.out.println(this.name); //编译错误, this指代当前Baby对象
        }
    }
}

```

3. 匿名内部类：应用率高，掌握

- 何时用：若想创建一个派生类的对象，并且对象只创建一次，可以设计为匿名内部类，可以大大简化代码
- 注意：匿名内部类中不能修改外面局部变量的值
- 小面试题：
 - 问：内部类有独立的.class吗？
 - 答：有

```

package ooday04;
public class AnonInnerClassDemo {
    public static void main(String[] args) {
        //1) 创建了Inter的一个派生类，但是没有名字
        //2) 为该派生类创建了一个对象，名为o1，向上造型为Inter类型
        // ----new Inter(){};是在创建Inter的派生类的对象
        //3) 大括号中的为派生类的类体
        Inter o1 = new Inter(){};

        //1) 创建了Inter的一个派生类，但是没有名字
        //2) 为该派生类创建了一个对象，名为o2，向上造型为Inter类型
        //3) 大括号中的为派生类的类体
        Inter o2 = new Inter(){};

        int num = 5;
        num = 6;
        //1) 创建了InterInter的一个派生类，但是没有名字
        //2) 为该派生类创建了一个对象，名为o3，向上造型为InterInter类型
        //3) 大括号中的为派生类的类体
        InterInter o3 = new InterInter(){
            public void show(){
                system.out.println("showshow");
                //num = 6; //编译错误，匿名内部类中不能修改外面局部变量的值
                //因为该变量在此处会默认为final的
            }
        };
        o3.show();
    }
}

interface InterInter{
    void show();
}

```

```
}

interface Inter{
}
```

4. package和import:

- package: 声明包
 - 作用: 避免类的命名冲突
 - 规定: 同包中的类不能同名, 但不同包中的类可以同名。
 - 类的全称: 包名.类名。包名常常有层次结构
 - 建议: 包名所有字母都小写
- import: 导入类
 - 同包中的类可以直接访问, 但不同包中的类不能直接访问, 若想访问:
 - 先import导入类, 再访问类-----建议
 - 类的全称-----太繁琐、不建议

```
package day04;
import java.util.Scanner; //1
//Scanner的演示
public class ScannerDemo {
    public static void main(String[] args) {
        /*
            package java.util;
            class Scanner{
                Scanner(InputStream s){ ... }
                int nextInt(){ ... }
                double nextDouble(){ ... }

                import java.util.Scanner;
                Scanner scan = new Scanner(System.in);
                int age = scan.nextInt();
                double price = scan.nextDouble();
            }
        */
        Scanner scan = new Scanner(System.in); //2
        int age = scan.nextInt(); //3
        double price = scan.nextDouble(); //3
    }
}
```

补充:

1. 多态的实际应用:

- 将不同对象(狗、鱼、鸡)统一封装到超类数组(动物数组)中来访问, 实现代码复用
- 将超类型(Animal)作为参数或返回值类型, 传递或返回派生类(Dog/Fish/Chick)对象, 以扩大方法的应用范围(所有Animal), 实现代码复用

2. 隐式的引用:

- this: 指代当前对象

- `super`: 指代当前对象的超类对象
- 外部类名.`this`: 指代当前对象的外部类对象

3. 关键字顺序: 先`package`, 再`import`, 最后`class`

4. 明日单词:

- 1) `public`: 公开的
- 2) `private`: 私有的
- 3) `protected`: 受保护的
- 4) `final`: 最终的
- 5) `static`: 静态的
- 6) `enum`: 枚举
- 7) `season`: 季节
- 8) `PI`: 圆周率
- 9) `spring`: 春
- 10) `summer`: 夏
- 11) `autumn`: 秋
- 12) `winter`: 冬
- 13) `value`: 值